Laporan Bagian B: VPS Server Setup & Deploy Microservices



Jacko Luciano 18224101
Aisyah Az Zahra 18224121

Institut Teknologi Bandung
2026

Daftar Isi

Daftar Isi **2**
Daftar Tabel **3**
Daftar Gambar **4**
Revisi Bagian A: Setup Server (STB) 5
1. Daftar Revisi 5
2. Langkah Pengerjaan Revisi 5
2.1 Persiapan awal dan koneksi SSH 5
2.2 Menghapus paket apt yang tidak sesuai slide 6
2.3 Pengecekan Docker dan sumber daya 6
2.4 Instalasi MySQL dan phpMyAdmin lewat Docker 6
2.5 Verifikasi menyeluruh Bagian A 7
2.6 Login phpMyAdmin 9
3. Versi Perangkat Lunak Final 10
Bagian B: Membangun API Microservice 11
4. Tujuan 11
5. Rancangan dan Arsitektur 11
6. Langkah Pengerjaan 12
6.1 Struktur proyek dan push ke GitHub 12
6.2 Clone dan menjalankan di STB 13
6.3 Membuat akun superuser 14
6.4 Import skema koleksi 15
6.5 Verifikasi gateway dan data produk 16
6.6 Pengujian API dengan REST Client 17
6.7 Verifikasi lewat dashboard PocketBase 21
6.8 Monitoring container di aaPanel 23
6.9 Repository GitHub akhir 23
7. Analisis 24
8. Kesimpulan 25

Daftar Tabel

Tabel 1.1 Daftar Revisi.....3
Tabel 2.4.1 Spesifikasi instalasi MySQL dan phpMyAdmin di Docker..... 4
Tabel 2.5.1 Ringkasan hasil verifikasi Bagian A..... 6
Tabel 3.1 Perangkat lunak setelah revisi.....8
Tabel 5.1 Daftar Service..... 9
Tabel 6.2.1 Hasil docker compose ps..... 12
Tabel 6.6.1 Hasil pengujian sembilan request..... 15
Tabel 6.7.1 API Rules koleksi favorites.....20

Daftar Gambar

Gambar 2.3.1 Pengecekan disk dan memori setelah pembersihan paket, lalu reboot STB..... 6
Gambar 2.4.1 Menu Docker, One-Click Install, aaPanel: MySQL dan phpMyAdmin berstatus Running..... 7
Gambar 2.5.1 Hasil verifikasi bagian pertama: sistem, SSH, aaPanel, dan Docker..... 8
Gambar 2.5.2 Hasil verifikasi bagian kedua: container, port yang listen, dan tools..... 8
Gambar 2.6.1 Halaman login phpMyAdmin (server 192.168.137.30:13306, user root)..... 9
Gambar 2.6.2 Halaman utama phpMyAdmin setelah login berhasil..... 10
Gambar 6.1.1 hasil initial commit..... 12
Gambar 6.1.2 Repository GitHub katalogku-pocketbase setelah first commit..... 13
Gambar 6.2.1 Hasil docker compose up -d --build: image dibuat, volume dan network dibuat, container Started..... 14
Gambar 6.2.2 git clone repository dan hasil docker compose ps..... 14
Gambar 6.3.1 Pembuatan superuser admin@katalogku.id di product-service dan account-service, serta cek health gateway..... 15
Gambar 6.4.1 Import collections pada product-service (:8091): tampilan Side-by-side diff.. 16
Gambar 6.4.2 Import collections pada account-service (:8092): tampilan Side-by-side diff.. 16
Gambar 6.5.1 Hasil curl api health dan curl products..... 17
Gambar 6.6.1 Request 1: cek gateway..... 18
Gambar 6.6.2 Request 2: daftar produk..... 19
Gambar 6.6.3 Request 3: pencarian produk dengan filter..... 19
Gambar 6.6.4 Request 4: detail produk..... 19
Gambar 6.6.5 Request 5: pendaftaran akun..... 20
Gambar 6.6.6 Request 6: login dan token..... 20
Gambar 6.6.7 Request 7: tambah favorit..... 20
Gambar 6.6.8 Request 8: lihat favorit..... 21
Gambar 6.6.9 Request 9: hapus favorit (204 No Content)..... 21
Gambar 6.7.1 Dashboard product-service (:8091), Collections, products: lima record produk.. 22
Gambar 6.7.2 Dashboard account-service (:8092), favorites, tab API Rules..... 23
Gambar 6.8.1 Daftar container running di aaPanel..... 23
Gambar 6.9.1 Repository GitHub akhir..... 24

Revisi Bagian A: Setup Server (STB)

1. Daftar Revisi

Tabel 1.1 Daftar Revisi

Aspek	Laporan A sebelumnya	Setelah revisi
Web server dan database	Apache, MariaDB 11.8.6, PHP 8.5, dan phpMyAdmin dipasang lewat apt	Seluruh paket apt tersebut dihapus. MySQL 9.0.1 dan phpMyAdmin 5.2.1 dipasang sebagai container Docker lewat One-Click Install aaPanel
Port dan akses aaPanel	Teks menyebut port 10760, sedangkan screenshot menunjukkan 1122	Port 1122, security entrance /aapanel, username dan password kuat hasil perubahan lewat perintah bt
Docker	Versi 29.9.2, diuji dengan hello-world	Versi 29.9.3 dan Compose v5.7.0, dicek dengan docker --version dan docker compose version
Verifikasi	Dicek per layanan secara terpisah	Satu skrip verifikasi yang memeriksa sistem, SSH, aaPanel, Docker, container, port, dan tools
Versi perangkat lunak	Beberapa kolom masih placeholder	Versi final dicantumkan pada Tabel 3.1

2. Langkah Pengerjaan Revisi

2.1 Persiapan awal dan koneksi SSH

STB diakses dari laptop lewat SSH, lalu paket sistem diperbarui, zona waktu diatur ke Asia/Jakarta, dan alat bantu dipasang.

```
None
ssh root@192.168.137.30

apt update && apt upgrade -y
timedatectl set-timezone Asia/Jakarta
apt install -y git curl nano htop
```

2.2 Menghapus paket apt yang tidak sesuai slide

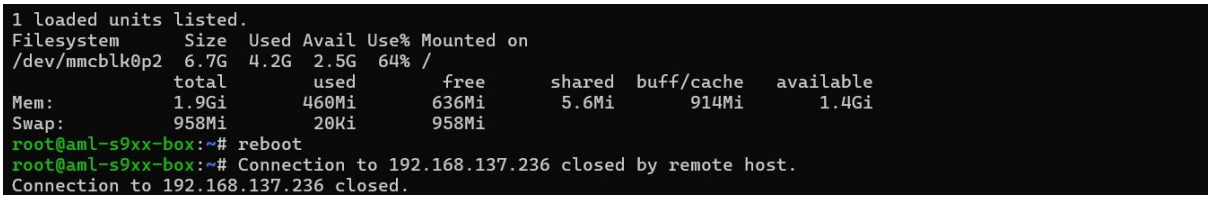
Apache, MariaDB, PHP, dan phpMyAdmin dari apt dihentikan, dihapus, lalu sisa paketnya dibersihkan. Pada STB ini belum ada data penting di database tersebut, sehingga backup tidak diperlukan. Penghapusan juga membebaskan RAM dan ruang SD card.

```
None
systemctl disable --now apache2 mariadb
apt purge -y 'apache2*' 'mariadb-*' 'php*' 'libapache2-mod-php*'
phpmyadmin
apt autoremove -y --purge && apt clean
```

2.3 Pengecekan Docker dan sumber daya

Setelah pembersihan, versi Docker, versi Docker Compose, dan sisa ruang disk dicek. STB juga di-reboot untuk memastikan layanan tetap normal setelah paket dihapus.

```
None
docker --version
docker compose version
df -h /
free -h
reboot
```



Gambar 2.3.1 Pengecekan disk dan memori setelah pembersihan paket, lalu reboot STB.

Pada Gambar 2.3.1, partisi root berukuran 6,7 GB dengan 4,2 GB terpakai (64%), dan memori 1,9 GB dengan 1,4 GB tersedia.

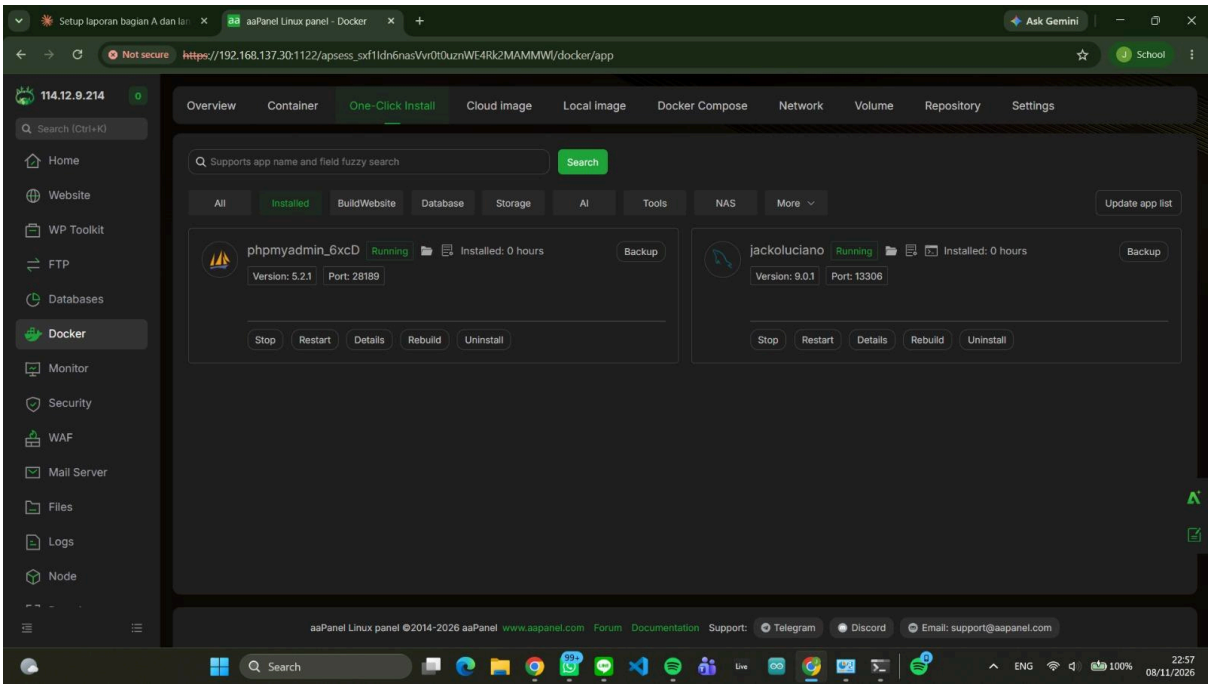
2.4 Instalasi MySQL dan phpMyAdmin lewat Docker

Pada panel aaPanel, buka menu Docker, lalu One-Click Install. MySQL dipasang lebih dulu, kemudian phpMyAdmin.

Tabel 2.4.1 Spesifikasi instalasi MySQL dan phpMyAdmin di Docker

Aplikasi	Versi	Port	Pengaturan

MySQL	9.0.1	13306	Root password kuat, memory limit 512 MB
phpMyAdmin	5.2.1	28189	Allow external access dicentang

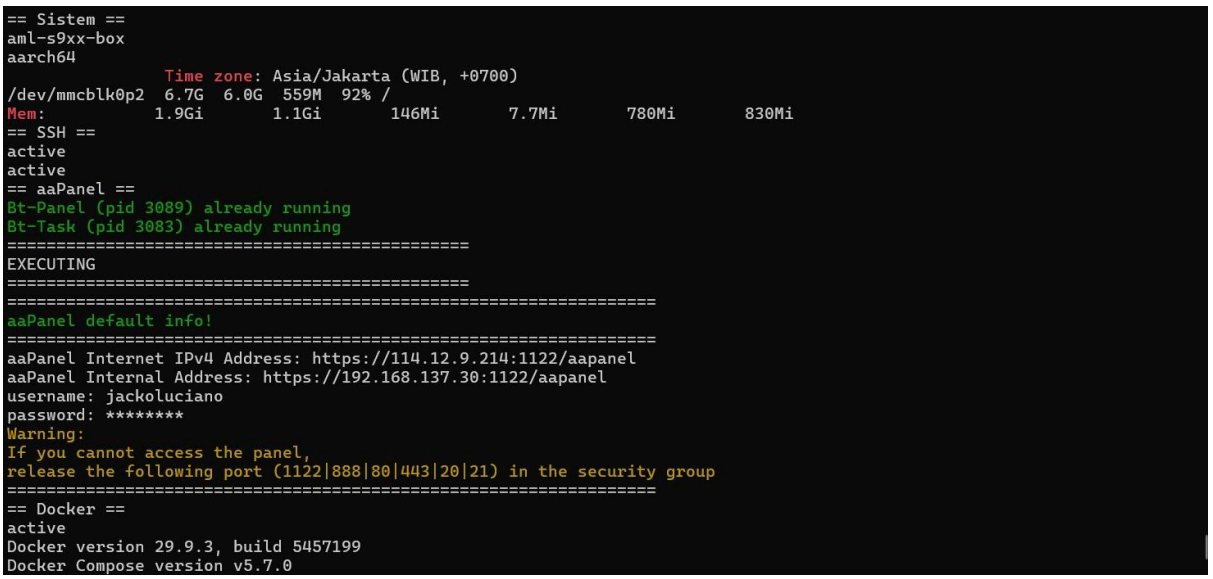


Gambar 2.4.1 Menu Docker, One-Click Install, aaPanel: MySQL dan phpMyAdmin berstatus Running.

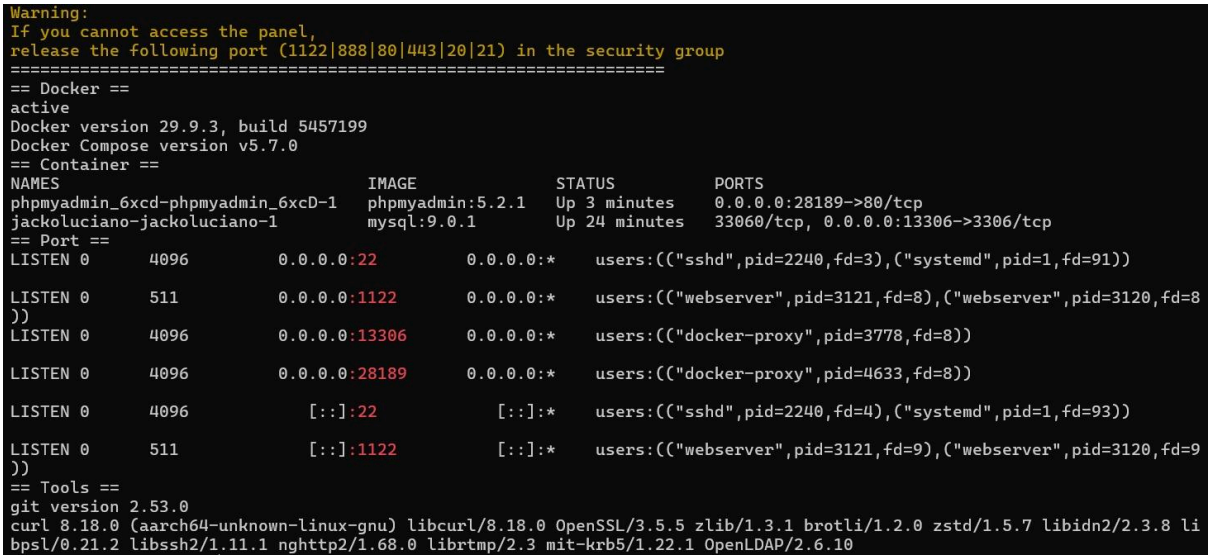
2.5 Verifikasi menyeluruh Bagian A

Seluruh komponen Bagian A diperiksa dengan satu skrip berikut.

```
None
echo "== Sistem ==";hostname; uname -m; timedatectl | grep "Time zone"; df -h / | tail -1; free -h | grep Mem
echo "== SSH ==";systemctl is-active ssh ssh.socket
echo "== aaPanel ==";bt status; bt 14
echo "== Docker ==";systemctl is-active docker; docker --version; docker compose version
echo "== Container ==";docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
echo "== Port ==";ss -tlnp | grep -E ':(22|1122|13306|28189)\b'
echo "== Tools ==";git --version; curl --version | head -1
```



Gambar 2.5.1 Hasil verifikasi bagian pertama: sistem, SSH, aaPanel, dan Docker.



Gambar 2.5.2 Hasil verifikasi bagian kedua: container, port yang listen, dan tools.

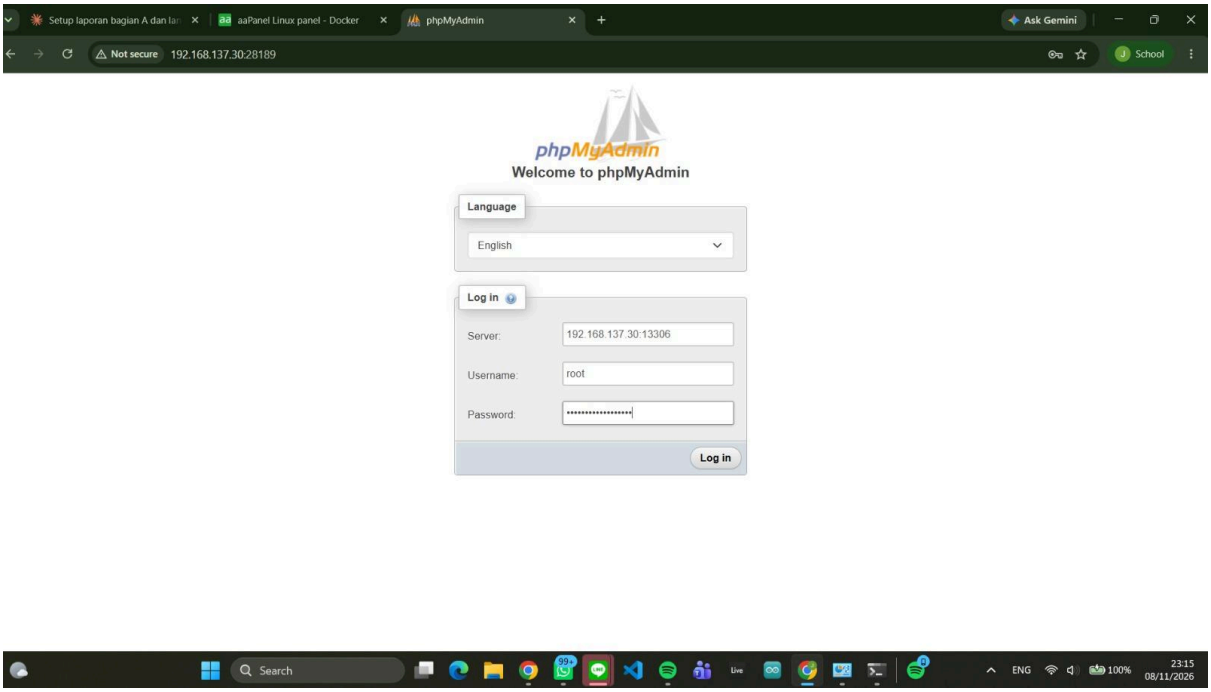
Tabel 2.5.1 Ringkasan hasil verifikasi Bagian A

Komponen	**Hasil**	**Status**
Hostname dan arsitektur	aml-s9xx-box, aarch64	OK
Zona waktu	Asia/Jakarta (WIB, +0700)	OK
SSH	ssh dan ssh.socket berstatus active	OK
aaPanel	Bt-Panel dan Bt-Task running, alamat https://192.168.137.30:1122/aapanel	OK

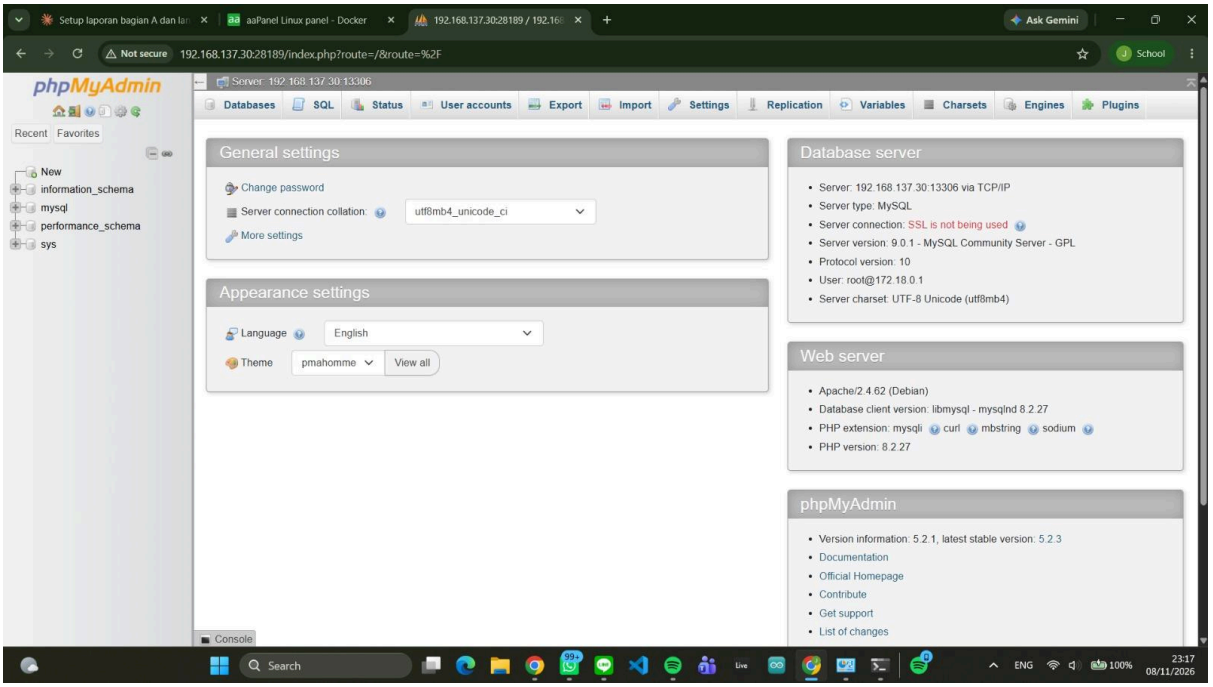
Docker	Active, versi 29.9.3, Compose v5.7.0	OK
Container	phpmyadmin:5.2.1 (port 28189) dan mysql:9.0.1 (port 13306) berstatus Up	OK
Port yang listen	22, 1122, 13306, 28189	OK
Tools	git 2.53.0, curl 8.18.0	OK

2.6 Login phpMyAdmin

phpMyAdmin dibuka di http://192.168.137.30:28189. Kolom server diisi 192.168.137.30:13306 dengan user root dan password MySQL dari langkah 2.4.



Gambar 2.6.1 Halaman login phpMyAdmin (server 192.168.137.30:13306, user root).



Gambar 2.6.2 Halaman utama phpMyAdmin setelah login berhasil

3. Versi Perangkat Lunak Final

Tabel 3.1 Perangkat lunak setelah revisi

Perangkat lunak	**Versi**	**Fungsi**
Armbian	community 26.11.0, kernel 6.18.57 (Aml-s9xx-box)	Sistem operasi STB
aaPanel	8.0.7	Panel manajemen server (port 1122)
Docker Engine	29.9.3	Platform kontainer
Docker Compose	v5.7.0	Mengelola banyak container
MySQL (Docker)	9.0.1	Database umum, port 13306
phpMyAdmin (Docker)	5.2.1	Antarmuka web database, port 28189
git	2.53.0	Mengambil kode dari GitHub
curl	8.18.0	Menguji API

Bagian B: Membangun API Microservice

4. Tujuan

1. Memecah backend aplikasi KatalogKu menjadi beberapa service kecil yang mandiri, dengan satu database per service.
2. Membangun API tanpa coding memakai PocketBase (versi 0.40.5) dengan Nginx sebagai API Gateway, semuanya dijalankan lewat Docker Compose.
3. Menyimpan konfigurasi ke GitHub, men-deploy-nya ke STB dengan git clone dan docker compose, lalu menguji API memakai curl dan REST Client.

5. Rancangan dan Arsitektur

Backend dibagi menjadi tiga container. Client tidak mengakses service secara langsung, melainkan hanya melalui gateway pada port 8080.

```
None
Client (REST Client / curl / aplikasi)
        |
   gateway (Nginx :8080)
     |                    |
/api/products        /api/auth, /api/favorites
     |                    |
product-service      account-service
(PocketBase #1)      (PocketBase #2)
SQLite: products     SQLite: users, favorites
dashboard :8091      dashboard :8092
```

Tabel 5.1 Daftar Service

Service	**Isi**	**Port server**	**Data**	**Prefix URL di gateway**
gateway (nginx:1.28-alpine)	Satu pintu masuk, meneruskan request ke service yang tepat	8080	tidak ada	/api/...
product-service (PocketBase)	Koleksi products	8091 (dashboard)	SQLite sendiri, volume product_data	/api/products
account-service (PocketBase)	Koleksi users (login) dan favorites	8092 (dashboard)	SQLite sendiri, volume account_data	/api/auth, /api/favorites

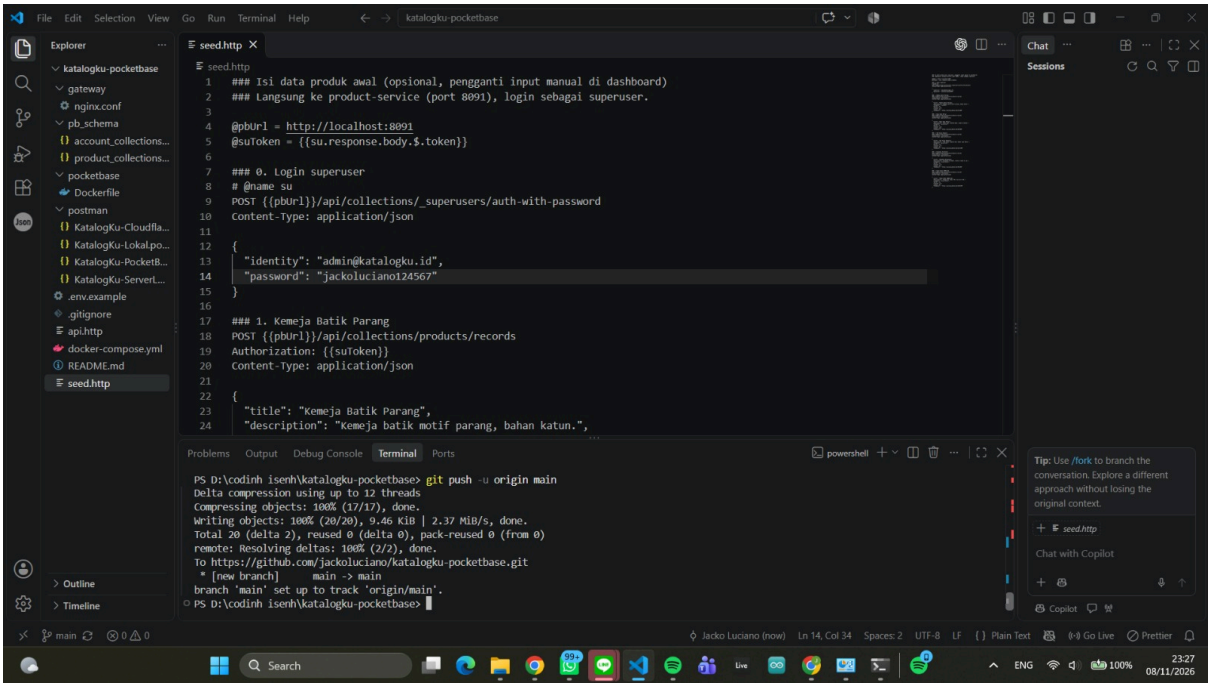
Prinsip yang dipakai adalah database-per-service. Tidak ada tabel yang dipakai bersama. Koleksi favorites hanya menyimpan product_id, sedangkan detail produk diambil dari product-service melalui API-nya.

6. Langkah Pengerjaan

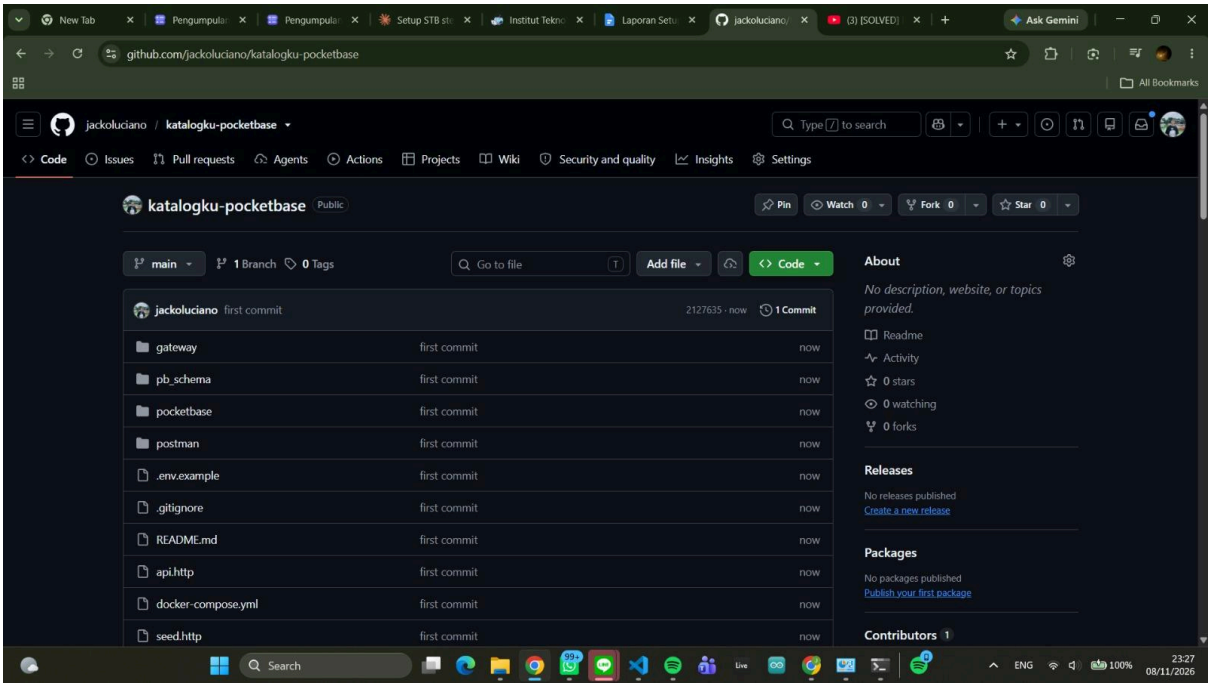
6.1 Struktur proyek dan push ke GitHub

Proyek katalogku-pocketbase berisi docker-compose.yml, folder pocketbase (Dockerfile), gateway (nginx.conf), pb_schema (skema koleksi untuk Import), postman, seed.http, api.http, .env.example, dan .gitignore. Isinya hanya konfigurasi tanpa kode program. File .env tidak ikut di-commit karena sudah masuk .gitignore, dan hanya .env.example yang disertakan.

Proyek dibuka di VS Code, lalu di-commit dan di-push ke branch main pada repository https://github.com/jackoluciano/katalogku-pocketbase.



Gambar 6.1.1 hasil initial commit



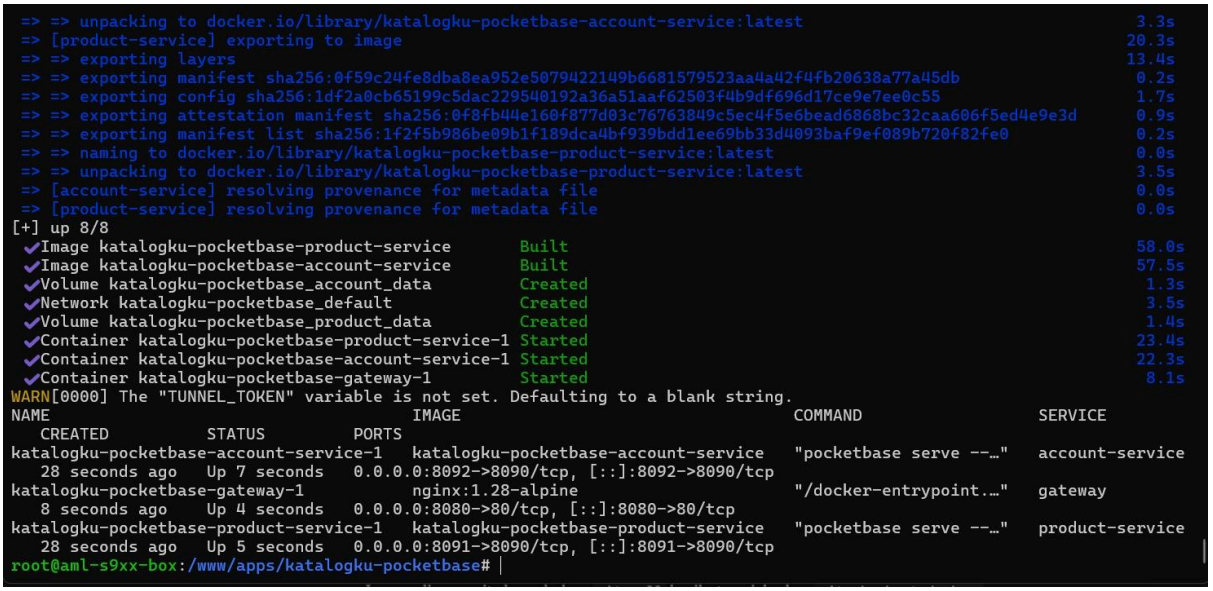
Gambar 6.1.2 Repository GitHub katalogku-pocketbase setelah first commit.

6.2 Clone dan menjalankan di STB

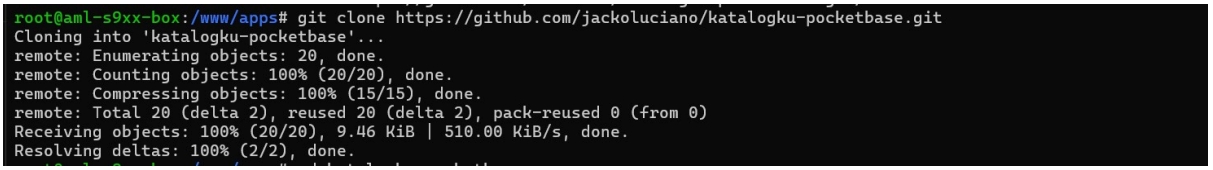
Pada STB, repository di-clone ke /www/apps, lalu seluruh service dibangun dan dijalankan.

```
None
mkdir -p /www/apps && cd /www/apps
git                                                    clone
https://github.com/jackoluciano/katalogku-pocketbase.git
cd katalogku-pocketbase
docker compose up -d --build
docker compose ps
```

Docker membangun dua image (product-service dan account-service, sekitar 58 detik), membuat network default serta volume product_data dan account_data, lalu menjalankan tiga container. Peringatan bahwa variabel TUNNEL_TOKEN belum diisi dapat diabaikan karena service tunnel bersifat opsional dan baru dipakai pada Bagian C.



Gambar 6.2.1 Hasil docker compose up -d --build: image dibuat, volume dan network dibuat, container Started.



Gambar 6.2.2 git clone repository dan hasil docker compose ps

Tabel 6.2.1 Hasil docker compose ps

Container	Image	Port
katalogku-pocketbase-gateway-1	nginx:1.28-alpine	8080 → 80
katalogku-pocketbase-product-service-1	PocketBase (build lokal)	8091 → 8090
katalogku-pocketbase-account-service-1	PocketBase (build lokal)	8092 → 8090

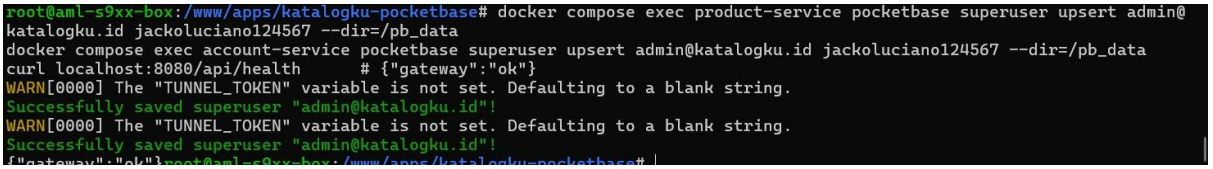
6.3 Membuat akun superuser

Setiap PocketBase memiliki database sendiri, sehingga akun superuser dibuat di kedua service. Password tidak ditulis di laporan.

```
None
docker compose exec product-service pocketbase superuser upsert admin@katalogku.id PASSWORD_KAMU --dir=/pb_data
```

```
docker compose exec account-service pocketbase superuser upsert
admin@katalogku.id PASSWORD_KAMU --dir=/pb_data
curl localhost:8080/api/health    # {"gateway":"ok"}
```

Kedua perintah menampilkan pesan Successfully saved superuser, dan endpoint health pada gateway membalas {"gateway":"ok"}.



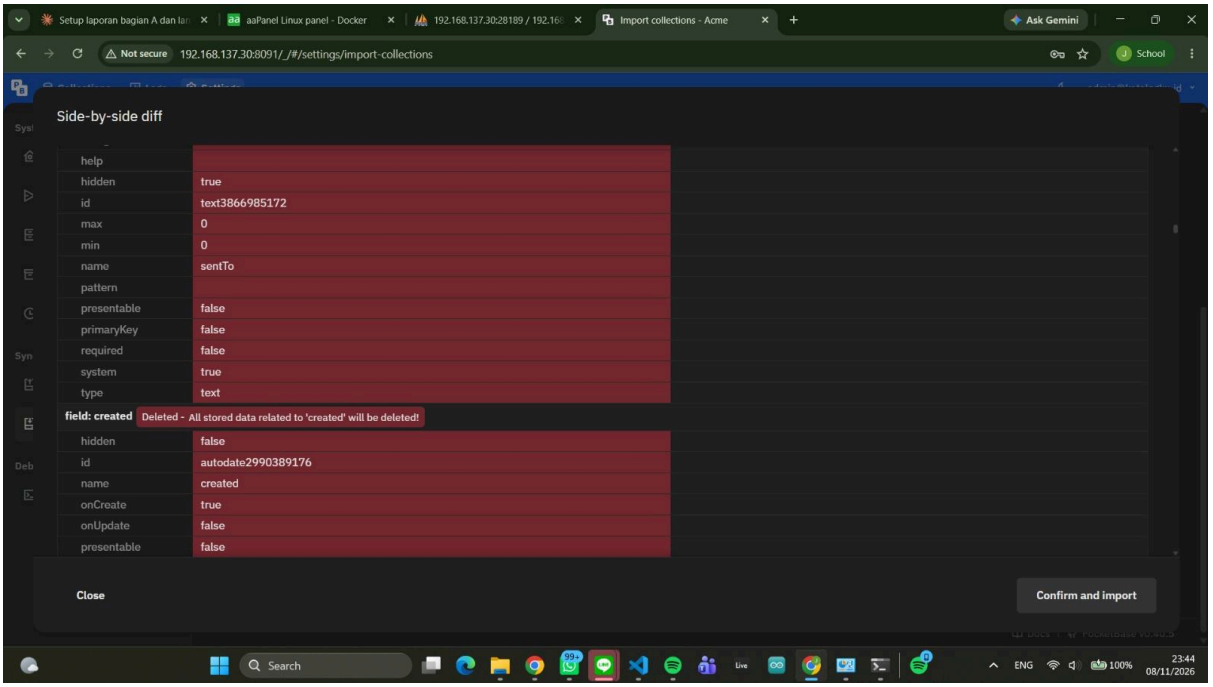
Gambar 6.3.1 Pembuatan superuser admin@katalogku.id di product-service dan account-service, serta cek health gateway.

6.4 Import skema koleksi

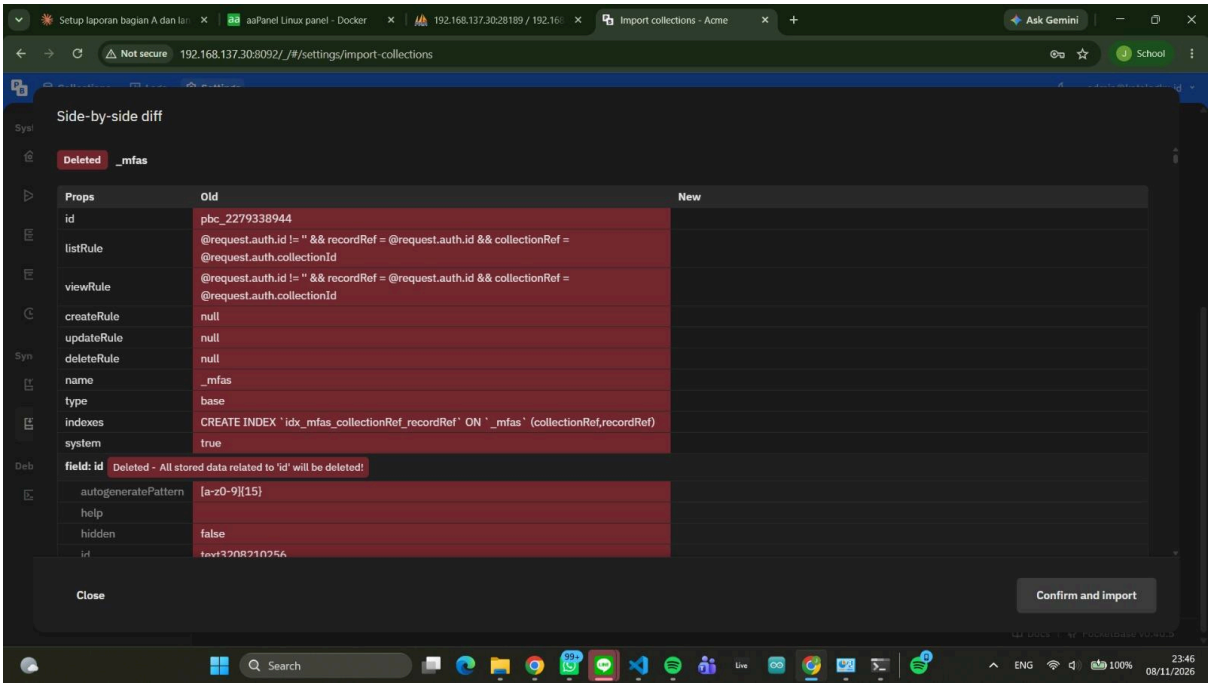
Koleksi dan API Rules dibuat lewat fitur Import collections, tanpa membuatnya manual satu per satu.

1. Buka http://192.168.137.30:8091/_/ dan login dengan admin@katalogku.id.
2. Buka Settings, lalu Import collections, lalu Load from JSON file, dan pilih pb_schema/product_collections.json.
3. Klik Review, periksa tampilan side-by-side diff, lalu klik Confirm and import.
4. Ulangi langkah yang sama di http://192.168.137.30:8092/_/ dengan pb_schema/account_collections.json.

Tampilan diff memuat beberapa entri bertanda Deleted untuk koleksi sistem bawaan PocketBase (misalnya _mfas) karena koleksi tersebut tidak ada di file JSON. Setelah import, koleksi sistem itu tetap ada, seperti terlihat pada Gambar 6.7.1.



Gambar 6.4.1 Import collections pada product-service (:8091): tampilan Side-by-side diff.



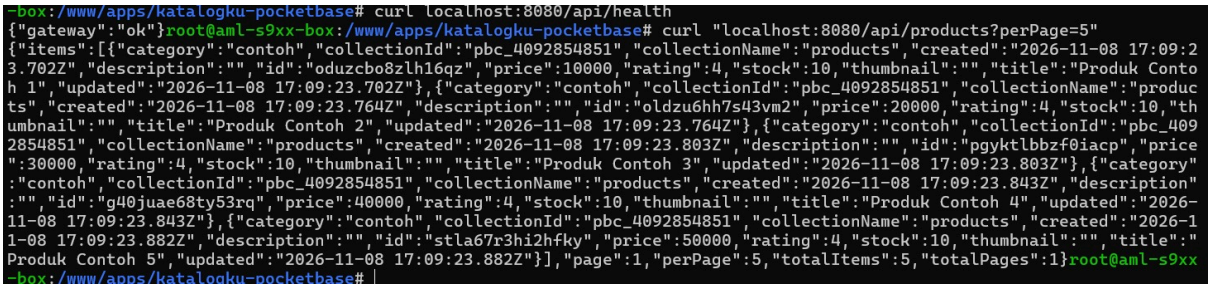
Gambar 6.4.2 Import collections pada account-service (:8092): tampilan Side-by-side diff.

6.5 Verifikasi gateway dan data produk

Gateway dan product-service dicek dari STB dengan curl.

```
None
curl localhost:8080/api/health
curl "localhost:8080/api/products?perPage=5"
```

Endpoint health membalas {"gateway":"ok"}. Endpoint products mengembalikan lima produk (Produk Contoh 1 sampai 5, kategori contoh, harga 10000 sampai 50000, rating 4, stok 10) dengan page 1, perPage 5, totalItems 5, dan totalPages 1. Hasil ini menunjukkan request lewat gateway sampai ke product-service dan data tersimpan di database miliknya.



Gambar 6.5.1 Hasil curl api health dan curl products

6.6 Pengujian API dengan REST Client

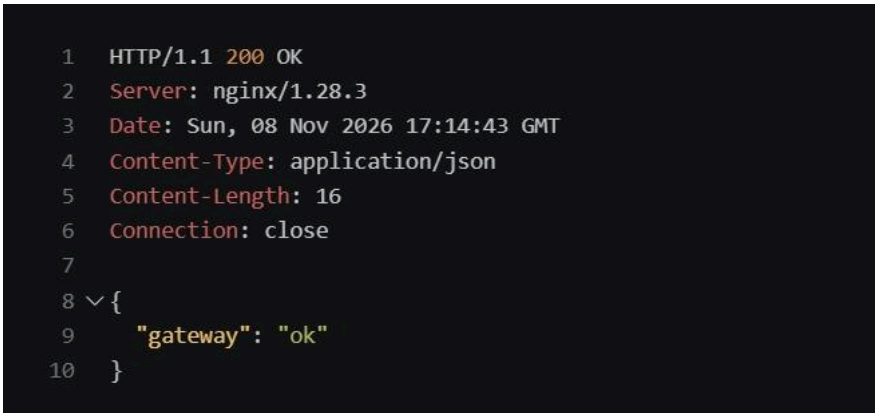
Pengujian dilakukan dari laptop memakai ekstensi REST Client di VS Code.

1. Buka folder katalogku-pocketbase di VS Code (ekstensi REST Client sudah terpasang).
2. Buka file api.http.
3. Ubah baris pertama menjadi `@baseUrl = http://192.168.137.30:8080/api`. Port yang dipakai adalah 8080 (gateway), bukan 8091 seperti pada seed.http.
4. Klik Send Request pada `###` 1, lanjut `###` 2 sampai `###` 9, satu per satu dan berurutan.
5. Periksa respons di panel kanan dan pastikan status HTTP 200 OK (atau 204 untuk penghapusan).

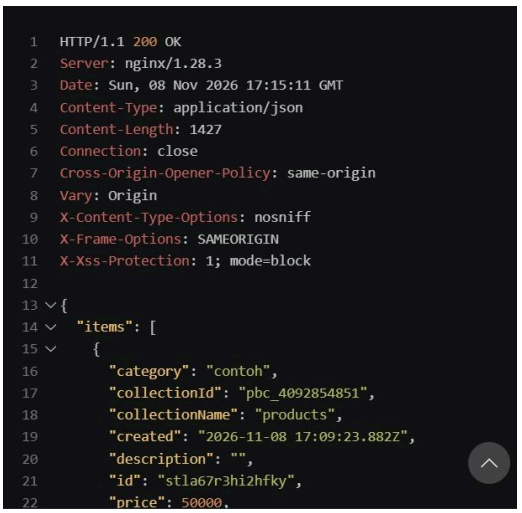
Tabel 6.6.1 Hasil pengujian sembilan request

No	Request	Endpoint	Hasil	Gambar
1	Cek gateway	GET /api/health	200, {"gateway":"ok"}	6.6.1
2	Daftar produk	GET /api/products?perPage=5&sort=-created	200, daftar produk dalam items	6.6.2

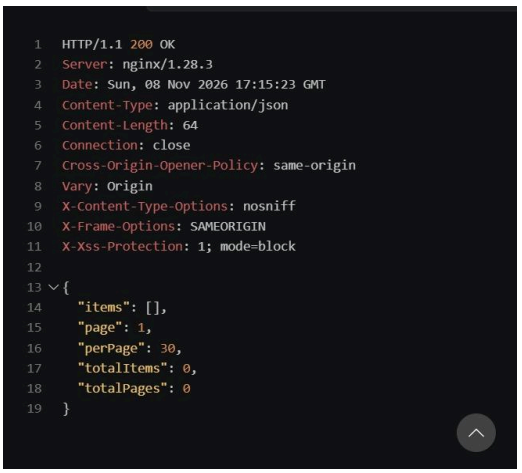
3	Cari produk	GET /api/products?filter=title ~'kopi'	200, items kosong (totalItems 0) karena belum ada produk berjudul kopi	6.6.3
4	Detail produk	GET /api/products/:id	200, satu produk	6.6.4
5	Daftar akun	POST /api/auth/records	200, record user baru pada koleksi users	6.6.5
6	Login	POST /api/auth/auth-with-password	200, token dan record user	6.6.6
7	Tambah favorit	POST /api/favorites (Bearer token)	200, record favorites berisi user dan product_id	6.6.7
8	Lihat favorit	GET /api/favorites (Bearer token)	200, items berisi favorit milik user	6.6.8
9	Hapus favorit	DELETE /api/favorites/:id (Bearer token)	204 No Content	6.6.9



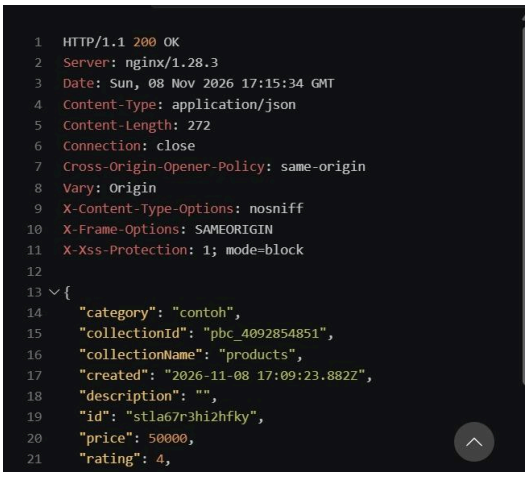
Gambar 6.6.1 Request 1: cek gateway.



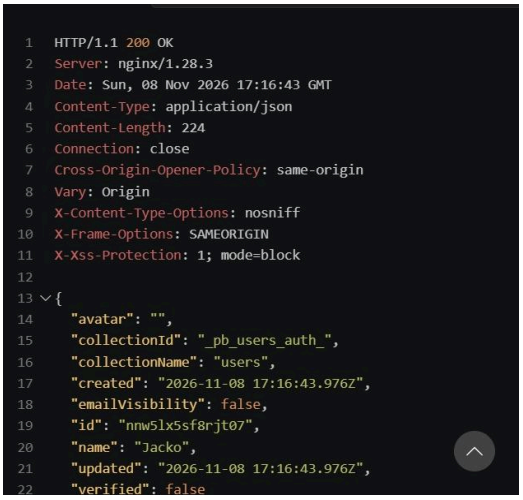
Gambar 6.6.2 Request 2: daftar produk.



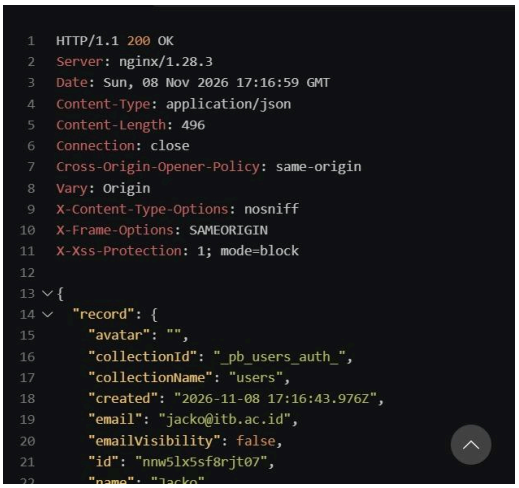
Gambar 6.6.3 Request 3: pencarian produk dengan filter.



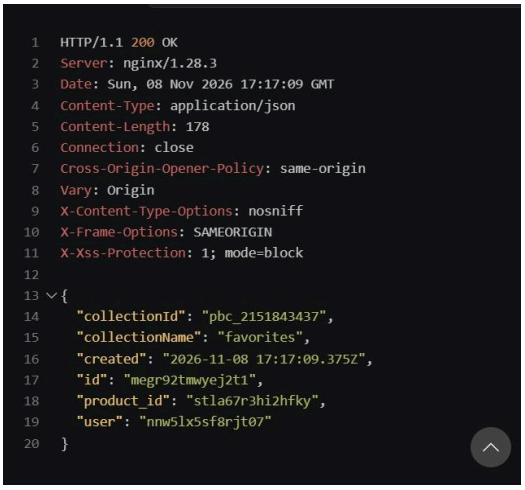
Gambar 6.6.4 Request 4: detail produk.



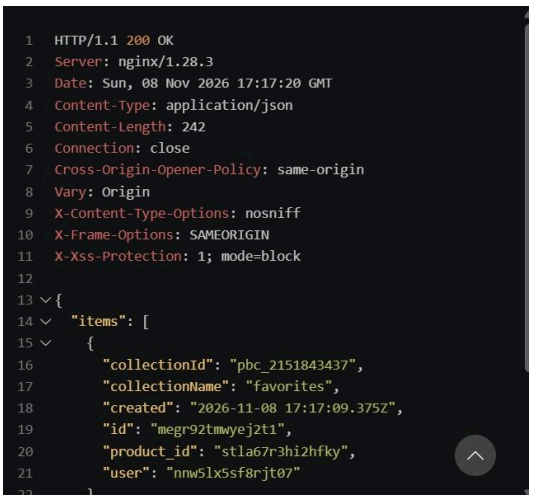
Gambar 6.6.5 Request 5: pendaftaran akun.



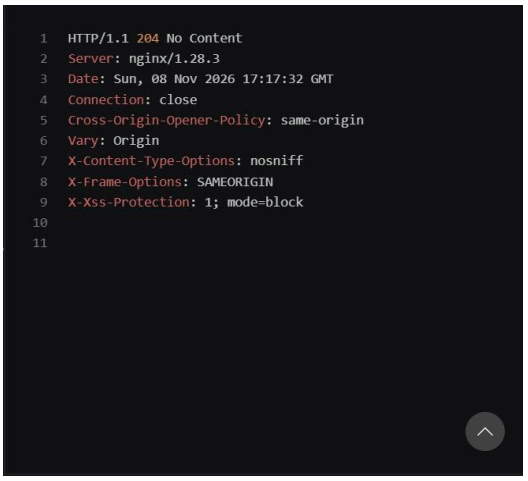
Gambar 6.6.6 Request 6: login dan token.



Gambar 6.6.7 Request 7: tambah favorit.



Gambar 6.6.8 Request 8: lihat favorit.

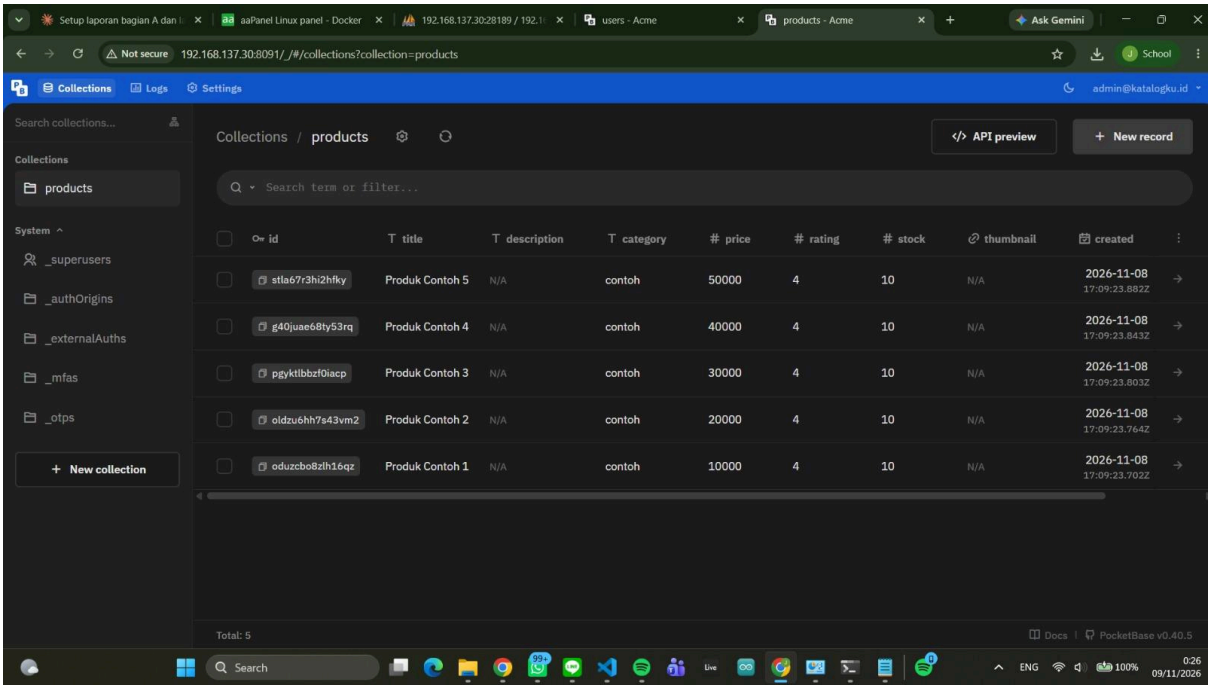


Gambar 6.6.9 Request 9: hapus favorit (204 No Content).

Alur pengujian ini sekaligus membuktikan alur favorit pada arsitektur: login menghasilkan token, token dikirim ke gateway, gateway meneruskannya ke account-service, lalu API Rules memeriksa kepemilikan data.

6.7 Verifikasi lewat dashboard PocketBase

Data dan aturan akses juga diperiksa langsung di dashboard masing-masing service.

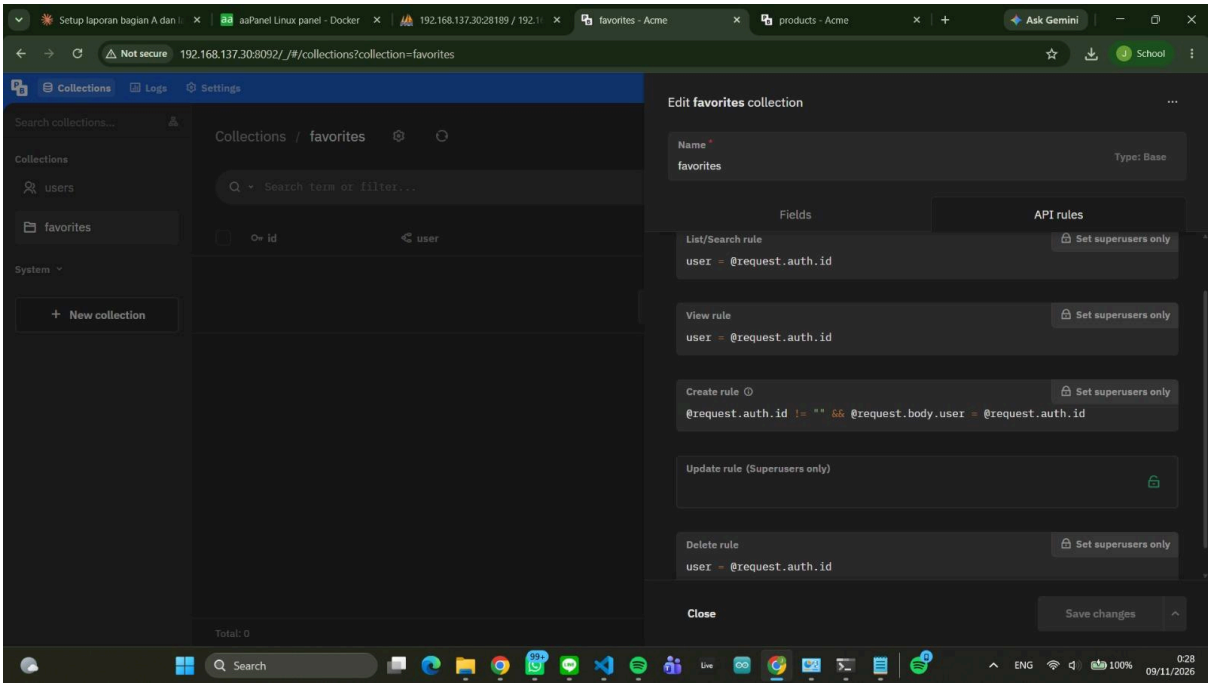


Gambar 6.7.1 Dashboard product-service (:8091), Collections, products: lima record produk.

Pada account-service, API Rules koleksi favorites dibatasi agar setiap user hanya mengakses datanya sendiri.

Tabel 6.7.1 API Rules koleksi favorites

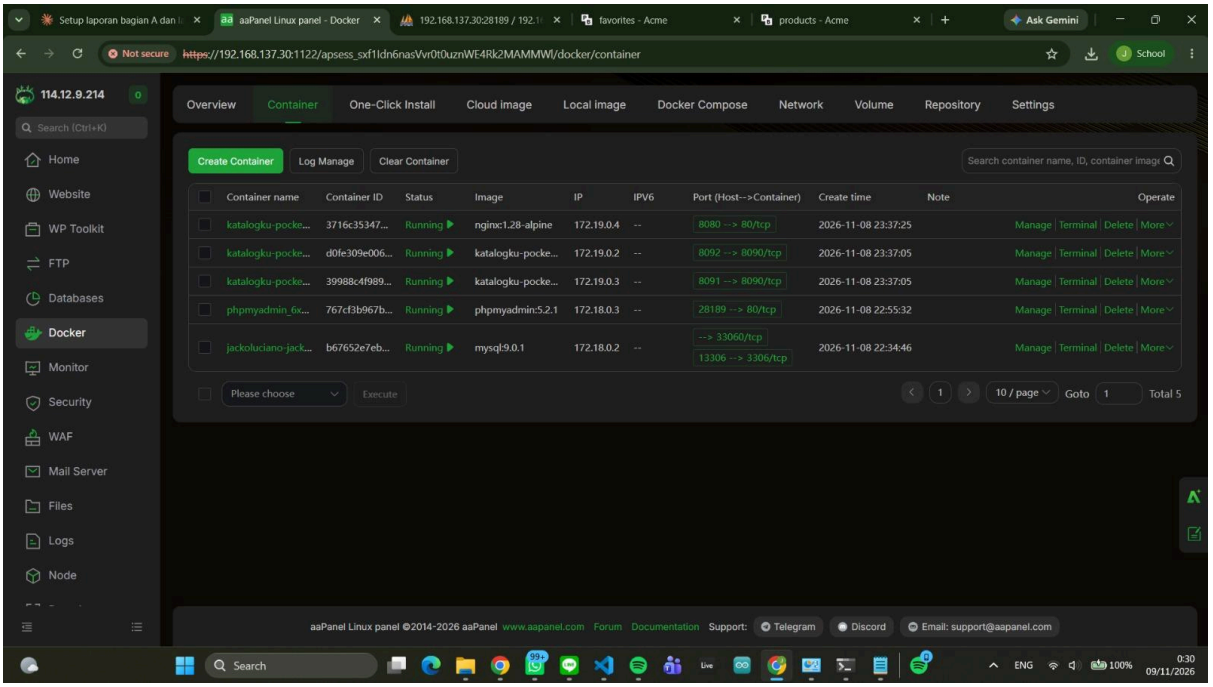
Rule	**Isi**	**Artinya**
List / Search	user = @request.auth.id	User hanya melihat daftar favorit miliknya
View	user = @request.auth.id	User hanya membuka favorit miliknya
Create	@request.auth.id != "" && @request.body.user = @request.auth.id	Harus login dan hanya boleh membuat favorit atas nama sendiri
Update	terkunci (superuser only)	Favorit tidak dapat diubah
Delete	user = @request.auth.id	User hanya menghapus favorit miliknya



Gambar 6.7.2 Dashboard account-service (:8092), favorites, tab API Rules.

6.8 Monitoring container di aaPanel

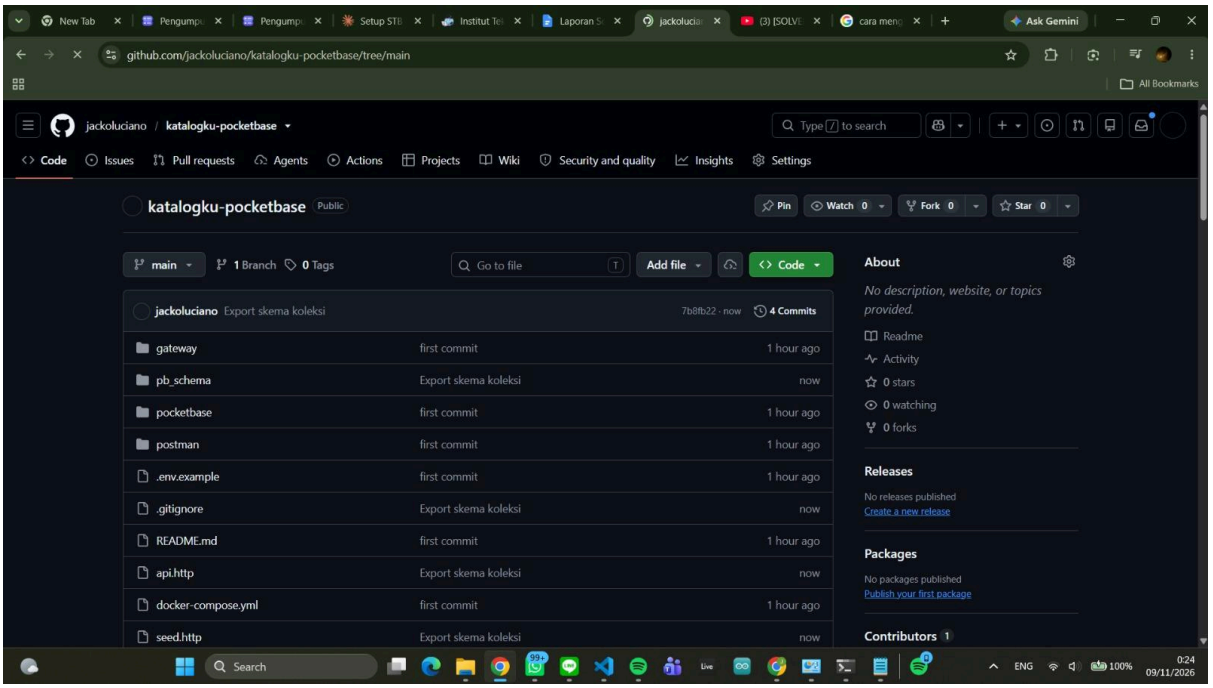
Kelima container (tiga container microservice, MySQL, dan phpMyAdmin) dapat dipantau di aaPanel pada menu Docker, Container.



Gambar 6.8.1 Daftar container running di aaPanel

6.9 Repository GitHub akhir

Setelah pengujian selesai, skema koleksi hasil perubahan di dashboard diekspor lewat Settings, Export collections, dan disimpan ke folder pb_schema agar ikut dikelola Git. Seluruh password contoh dikembalikan menjadi placeholder sebelum commit akhir, dan file .env tidak ikut di-commit. Repository akhir berisi gateway, pb_schema (hasil export), pocketbase, postman, docker-compose.yml, api.http, seed.http, .env.example, .gitignore, dan README.md.



Gambar 6.9.1 Repository GitHub akhir

7. Analisis

1. **Database-per-service.** product-service dan account-service memakai PocketBase yang sama tetapi dengan volume dan database SQLite yang terpisah. Perubahan skema di salah satu service tidak memengaruhi service lain, dan data produk tidak tercampur dengan data akun.
2. **Peran API Gateway.** Aplikasi hanya perlu mengenal satu alamat (http://IP_SERVER:8080/api). Nginx memetakan /api/products ke product-service serta /api/auth dan /api/favorites ke account-service. Service di belakangnya bisa dipindah atau diganti tanpa mengubah client.
3. **Keamanan lewat API Rules.** Koleksi products terbuka untuk dibaca semua orang tetapi hanya superuser yang dapat mengubahnya. Koleksi favorites dibatasi per pemilik sehingga satu user tidak dapat melihat atau menghapus favorit user lain.
4. **Alur kerja berbasis Git.** Server hanya menjalankan kode hasil git clone. Perubahan dilakukan di laptop, di-push ke GitHub, lalu di-pull di server. Skema koleksi ikut tersimpan lewat Export collections sehingga lingkungan dapat dibuat ulang di mesin lain dengan Import collections.

5. **Komposisi data di sisi client.** Koleksi favorites hanya menyimpan product_id. Detail produk diambil dari product-service lewat endpoint /api/products/:id, sehingga tidak ada data produk yang diduplikasi di account-service.

8. Kesimpulan

Backend KatalogKu berhasil dipecah menjadi tiga container (gateway, product-service, account-service) dan dijalankan di STB dengan Docker Compose. Seluruh endpoint yang dirancang (health, daftar, pencarian, dan detail produk, pendaftaran, login, serta tambah, lihat, dan hapus favorit) menghasilkan respons sesuai kontrak API. API Rules memastikan produk hanya bisa diubah admin dan favorit hanya bisa diakses pemiliknya. Konfigurasi, skema koleksi, dan gateway tersimpan di repository GitHub katalogku-pocketbase tanpa file .env, sehingga lingkungan yang sama dapat dibangun ulang dengan git clone dan docker compose up.

Repository: https://github.com/jackoluciano/katalogku-pocketbase